

GateSim: A Two-Phase Synchronous Frontier Architecture for Affected-Region Logic Simulation

Arpit Shinde, Harsha V

Department of Computer Science and Engineering

Indian Institute of Information Technology Design and Manufacturing Kurnool
Kurnool, India

Abstract—Digital Logic Design (DLD) and Computer Organization and Architecture (COA) are closely related subjects, which involve implementing large hierarchical logic structures through simulations. Some existing simulators may not handle sequential loops, and unnecessarily execute redundant computation. This paper presents GateSim, a web-based digital logic simulator focused on two-phase affected-region propagation architecture. A change is propagated only through the region it can affect, avoiding evaluation of unaffected regions. The same mechanism runs inside reusable custom components, enabling larger circuits. Synchronous frontiers also expose opportunities for parallel evaluation.

Index Terms—Digital Logic, Circuit Simulation, Computer Organization, Educational Tools, Web-based Simulation

I. INTRODUCTION

GateSim¹ is a web-based logic simulator that uses a two-phase affected region propagation architecture to evaluate only the portion of circuit affected by change. It uses synchronous frontiers to separate evaluation from state commitment, which exposes the nodes as a unit of parallel computation. The simulator is also designed to bridge the gap between DLD and COA learning workflow through hierarchical circuit construction.

Existing simulators use different strategies for simulation. Logisim-Evolution [3], CircuitVerse [1] rely on event-driven queues. In our testing of Logisim-Evolution v5.0.0 [3], a NAND-gate implementation of a JK flip-flop did not produce the expected state of circuit, regardless of input states. COLDVL [2] combines event-driven simulation with topologically ordered levelised simulation. It introduces ordered reconstruction to handle non-convergence in feedback-containing circuits, and amalgamated simulation in linear time for circuits conforming to the Huffman model.

As circuits become larger, recomputing unaffected regions lead to unnecessary work. Selective-trace and event-driven simulation techniques address this by computing only those components which are affected by signal change [4]. GateSim adopts this principle. Furthermore, in queue-based event driven architectures like CircuitVerse, the circuit state is updated each time a queued element is evaluated. Thus, the queue itself does not explicitly identify a set of elements that can be evaluated simultaneously. In contrast, GateSim groups affected nodes into sets whose elements can be evaluated simultaneously, enabling parallel computation across the set.

II. DESIGN

A. Circuit Representation

A circuit is represented as a directed graph stored in an array. Each node has a type, a value array which stores the current state of node, a list of input references, and a list of output references. Unconnected input pins are treated as logic 0, and signals are two-valued (true/false).

B. Affected-Region Propagation

Let v denote a node and $Out(v)$ denote the list of nodes to which v is connected. A “frontier” F_i is a list which contains the nodes to be evaluated at i^{th} step. Let f_v and s_v denote the computed value from its input sources and current value of node v , respectively. When a source node s is toggled, the propagation starts from 0^{th} step as:

$$F_0 = Out(s). \quad (1)$$

Each propagation step has two phases.

Phase 1 (evaluate). Every node in the current frontier F_i is evaluated independently using the same previously committed state of circuit, allowing the evaluations to be performed in parallel. No value is written during this phase for primitive gates. The set of nodes whose computed value differs from its previous value is

$$C_i = \{v \in F_i \mid f_v \neq s_v\}. \quad (2)$$

Phase 2 (commit). All new values are written together. The next frontier is then formed from the outputs of the nodes that changed:

$$F_{i+1} = \bigcup_{u \in C_i} Out(u). \quad (3)$$

Propagation ends when F_i becomes empty. A node that is evaluated but does not change its value does not let the signal propagate further along its outputs. Propagation follows parent-to-child connectivity from changed source through outputs of updated nodes, rather than graph storage order.

C. Performance

For an affected acyclic region with A nodes and maximum propagation depth D , total evaluations are bounded by $\mathcal{O}(A \cdot D)$. A reconvergent circuit like Table 1, experiment 3 demonstrates that this bound can reach $\mathcal{O}(A^2)$. If the region contains a feedback loop, D is not determined by path length and no bound is claimed. Stable loops stop when its outputs stop changing. A loop that is not stable keeps producing frontiers. The number of propagation steps are thus limited

¹GateSim is available online at: <https://arpit-shinde.github.io/GateSim/>

TABLE I
PROPAGATION EVALUATION RESULTS FOR AFFECTED-REGION AND
RECONVERGENT PATH EXPERIMENTS.

Experiment	Affected Gates	Total Gates	Propagation Steps	Number of Evaluations	Avg. Time (μ s)
1A	64	576	65	65	300.00
1B	64	1088	65	65	300.00
2A	128	1088	129	129	345.00
2B	256	1088	257	257	500.00
2C	1024	1088	1025	1025	1055.00
3A	5	—	5	15	—
3B	7	—	7	28	—
3C	9	—	9	45	—
3D	11	—	11	66	—

to $\max(N + 1, K)$ to terminate, where N is total number of nodes in the circuit, K is a hardcoded limit and a warning is sent if the frontier is still non-empty at the limit.

D. Settling

After structural edits, GateSim repeatedly evaluates the full circuit in storage order until no values change or an iteration limit is reached. This establishes an initial state rather than a correct state, and subsequent user-driven changes use the affected-region propagation.

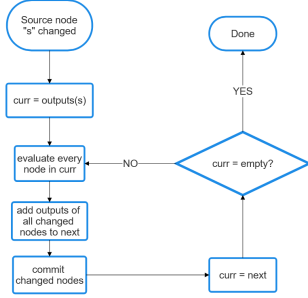


Fig. 1. Flowchart for algorithm.

III. EVALUATION

The evaluation was done across the following domains:

Affected-region propagation : From Table I, in Experiment 1A, toggle t_1 was fed to a linear chain of 64 NOT gates and another toggle fed an independent network of 512 NOT gates. In 1B, circuit from 1A was extended by an additional independent network of 512 NOT gates. In both cases, only t_1 was toggled, and output was observed in an additional output node. The number of propagation steps remained strictly same while average propagation time over 20 sampled tests remained approximately same, demonstrating that only the affected (64 NOT gate network) was evaluated. In 2A, 2B, and 2C, the total size of circuit was kept constant while affected region was progressively increased, resulting in an approximately linear increase in propagation time. In Experiment 3, a common input drove one input of each of XOR gate. The second input of the first XOR gate was driven by an independent toggle, while the output of each XOR gate was connected to the second input of the subsequent XOR gate. No additional output node

was used here. Toggling the common input produces frontiers $F_i = \{n_{i+1}, \dots, n_n\}$, with frontier sizes decreasing from n to 1. Hence, the total number of node evaluations is

$$Q = \sum_{i=0}^{n-1} |F_i| = n + (n-1) + \dots + 1 = \frac{n(n+1)}{2} = \mathcal{O}(n^2),$$

consistent with $A = n$ and $D = n$. “Total Gates” represent the number of logical gates in the circuit. Wires are nodes in GateSim, but the Total Gates, Propagation Steps, and Evaluations columns count logic gates only.

Parallelization Potential : An OpenGL compute-shader was written which implements the two-phase frontier logic. It was benchmarked on a circuit shown in Fig. 2 containing 2^{20} nodes per frontier across 8 propagation layers, totaling 8,388,608 node evaluations. The program ran on an Intel Core i5-13420H (-O2) and RTX 3050 6GB. Averaged over five trials, the CPU and GPU execution times were 68.47 ms and 8.88 ms, respectively, demonstrating a $7.7\times$ speedup of GPU over CPU. This program is separate implementation, not the GateSim engine, and demonstrates that circuits with large fan-out produce wide frontiers, providing parallel computation opportunity.

IV. CONCLUSION, LIMITATIONS AND FUTURE WORK

Instead of re-evaluating the entire circuit every time a change occurs, GateSim uses connectivity-driven model. It is based on two-phase evaluate and commit system, ensuring the runtime propagation independent of graph storage order and allows hierarchical custom components to apply the same mechanism.

The design has few limitations as of now. The settling pass is order dependent and may exhibit inaccurate circuit state. Signals are two-valued, so unknown initial states, high impedance, and bus contention are not modelled. It does not distinguish race-around conditions from other forms of non-convergent feedback; such behavior is detected generically when propagation exceeds the step limit.

Future work includes reducing worst-case quadratic evaluation in reconvergent circuits, an unknown-value or case-analysis mechanism inspired by COLDVL, race-around warnings, richer sequential, memory components, additional hierarchical COA structures, a controlled classroom workflow and evaluation of GateSim as a DLD-to-COA learning environment.

REFERENCES

- [1] CircuitVerse, “CircuitVerse simulator source code (simulation engine),” open-source repository. [Online]. Available: <https://github.com/CircuitVerse/CircuitVerse>
- [2] G. Roy, D. Ghosh, and C. Mandal, “COLDVL: a virtual laboratory tool with novel features to support learning in logic design and computer organisation,” *Journal of Computers in Education*, vol. 4, pp. 461–490, 2017, doi: 10.1007/s40692-017-0091-8.
- [3] LogiSim-Evolution, “LogiSim-Evolution Releases,” 2026. [Online]. Available: <https://github.com/logisim-evolution/logisim-evolution/releases>
- [4] Ernst G. Ulrich. 1969. Exclusive simulation of activity in digital networks. *Commun. ACM* 12, 2 (Feb. 1969), 102–110. <https://doi.org/10.1145/362848.362870>